



國立台灣科技大學

資訊工程系

碩士學位論文

筆觸與色塊分離之動畫向量化

Cartoon Animation Vectorization Based on Stroke and
Region Separation

黃亮軒

M10015098

指導教授：姚智原 博士

中華民國一零三年十月三日

中文摘要

在向量化技術中，影像向量化通常會將原影像分解成大量顏色區塊來組成，將區塊內的顏色生成顏色模型產生平滑的填色，而動畫向量化的難點在，分析動態與靜態的區塊，但在維持靜態場景的連續性是關鍵，因為所有的演算法都會因為細微誤差得到不同結果，導致結果閃動。

在本論文我們將向量化演算法分成兩部份，一是影像向量化部份，二是動畫連續性部份，本論文使用了有別於傳統的色塊向量化，使用了筆觸分離的概念，將色塊與筆觸分開處理並向量化，以達到更銳利的筆觸線條。

而動畫分成靜態場景與動態場景的向量化，我們提出使用關鍵幀來修正每幀三角化的區塊對應，來維持連續動畫每一幀的連續性，來分析動態區塊的部份，小區塊有誤差的部份再用前後幀的原始影像做修正，動態場景則是將每幀獨立向量化。

我們的研究成果能輸入一個動畫短片，並轉換成向量化動畫播放。

Abstract

This paper presents a vectorization system for cartoon animation. Our vectorization method is based on processing separately from stroke and region. We can generate outputs that are visually flicker-free and with clear stroke. We use image decomposition algorithm to do vectorization separately on stroke and region, and supporting color model for each region. To ensure temporal coherence in the output, we use region tracking algorithm for all frames to maintain region's color and shape. The contributions of this paper are: 1) individual handling for strokes during image decomposition process to produce results with sharpen strokes. 2) generating flicker-free results by maintaining region coherence for all frames.

Table of Contents

中文摘要	i
Abstract	ii
Table of Contents	iii
List of Figures	1
1 序論	1
1.1 研究背景與動機	1
1.2 論文貢獻	1
1.3 論文架構	3
2 相關研究	4
2.1 圖片向量化	4
2.2 動畫向量化	12
3 論文簡介	16
4 研究方法	18
4.1 影像預處理	18
4.2 動畫特徵截取	21
4.2.1 顏色邊界萃取	23
4.3 顏色特徵萃取	29

4.3.1	筆觸萃取	29
4.3.2	筆觸向量化	31
4.4	計算移動距陣	33
4.5	前景區塊萃取	37
4.6	動態場景之區塊對應	39
5	實驗結果	41
6	結論與未來展望	51
	References	56

Chapter 1 序論

1.1 研究背景與動機

卡通動畫，是一個結合大量藝術家產生的一個藝術作品，有著悠久的歷史，受到大人與小孩等廣大群眾的歡迎，卡通動畫也包含不同的風格，藝術家們利用不同粗細、深淺的筆觸，單調或豐富的顏色漸層，來展現每個作品的風格。

相較於點陣圖，向量圖有下列優點，需要的資訊較少，因此檔案大小通常比點陣圖還要小，檔案大小與解析度無關，圖形可以無限放大，並在邊界保持平滑，圖形的參數較容易修改，

一般卡通動畫通常是由漸層的色塊與乾淨的線條所組成，部份會混合有大量細節的背景，例如森林和都市，以點陣圖儲存的影片在放大觀賞時，會在顏色邊界與筆觸部份，會出現鋸齒狀的失真。向量圖有放大後線條依然乾淨，色塊邊界依然明確的優勢，因此，若能將老舊動畫成功的轉換成向量動畫，就能解決這些鋸齒狀失真，且不再有解析度的困擾，可在任意尺寸的顯示器上播放。

1.2 論文貢獻

- 卡通通常是用筆觸去構畫標明元素，再利用色塊增加光影及貼圖細節，我們提出將筆觸與色塊分離的卡通向量化方法，各自以最佳的表示方式記錄及生成，以達到忠實地呈現原始結果。
- 大部分的卡通影格包含大量的重複訊息，我們提出利用色塊在時續上的空間性與顏色相對距離來維持區塊時續連續性的方法，讓筆觸及色塊能在向量化動畫中穩定運作，維持對應區塊的連續性，以減少動畫的閃爍與跳動，並且

減少記憶體的使用量。

1.3 論文架構

本論文共分為六章，各章節分別如下：

第一章 序論 介紹研究動機與貢獻。

第二章 相關研究 概述向量圖的特性與向量化的相關研究。

第三章 論文簡介 介紹本論文的方法，包括整個動畫的向量化流程，單張影格的向量化過程與動畫連續性的維持過程。

第四章 研究方法 介紹論文流程與步驟，並解釋我們如何運用相關研究在本論文的演算法中，包含單張影像向量化與動畫向量化兩部份，與本論文所使用到的相關研究，包括Cheng09 [7]影像特徵萃取演算法以及WengCF05 [8]白平衡演算法。

第五章 實驗結果 針對一些動畫影像進行向量化，並且比較分析其向量化效果。

第六章 結論 介紹本論文的限制與未來可以發展改善的方向。

Chapter 2 相關研究

2.1 圖片向量化

參考主流的繪圖軟體的向量化功能，如圖2.1,2.2，從這些圖可以了解目前的市面軟體向量化多著重在同顏色區塊的分割，而切割的愈細小讓人感覺與原圖愈相近，但使用在動畫風格的影像上，會在顏色有漸層的部份有明顯的邊界。



(a) 真實影像向量化前

(b) 真實影像向量化後

Figure 2.1: 真實影像使用CorelDraw向量化比較



(a) 動畫影像向量化前

(b) 動畫影像向量化後

Figure 2.2: 動畫影像使用CorelDraw向量化比較

Lecot等人 [1]所提出的ARDECO系統使用線性與二次梯度來建立顏色模型，如圖2.3所示，此系統會先將輸入的影像依顏色分割出大量的區域，在顯著權重愈高

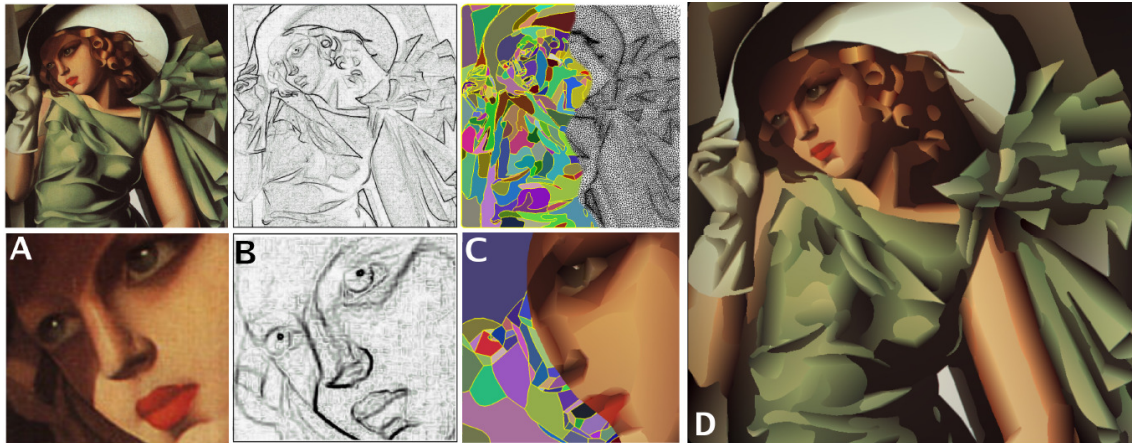


Figure 2.3: Lecot等人 [1]的ARDECO系統向量圖製作流程，原圖(A)，產生顯著權重(saliency weight)(B)，計算每個區塊的顏色模型並三角化(C)，得到最後結果(D) 引用自 [1]

的部份分裂出愈多區塊，並在每個區塊內透過線性系統搭配三種顏色模型來做輸出，這三種模型分別為固定、線性、二次函數，使用的時機由區塊內的顏色變化複雜度來決定。雖然各區塊內的顏色會近似原圖的顏色，但每個區塊之間的邊緣太過銳利，使得向量化後的結果有明顯的分割痕跡。

在目前常使用的繪圖軟體(例如：Corel Draw、illustrator等)則是引入了由Sun等人 [2]所提出的漸層網格(Gradient mesh)，如圖2.4所示，這種向量工具可以半自動化的協助美術人員設定每個網格上的顏色，而四角網格中間的顏色值便會平滑地內插四周的網格顏色。但若要將影像轉換成漸層網格，美術人員必須要有足夠的技巧及耐心，因為他們必須要精準地將網格的分佈位置對應到圖片上，而這需要花上大量的時間來完成。

Lai等人 [3]提出了一種以拓撲結構為基礎的漸層網格，並提出一個自動生成向量圖的方法。如圖2.5所示，此方法會將圖片三角化，再分析圖片的幾何結構將其參數化，最後透過這些資訊來生成漸層網格，但生成網格需要大量的時間，且需要大量的省存空間。

Zicheng等人 [4]提出了以細化三角面為基礎的演算法，來對真實影像向量化，

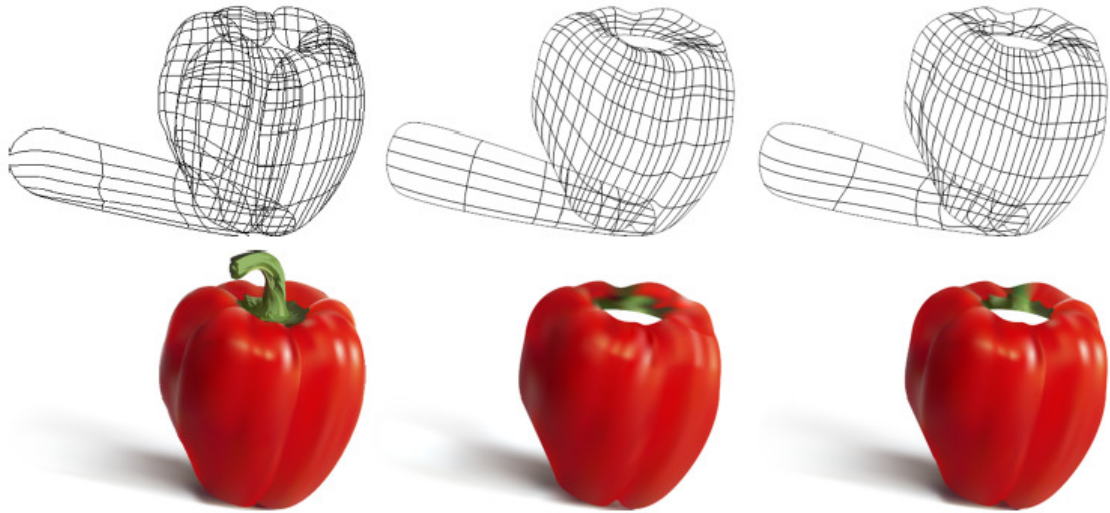


Figure 2.4: Sun等人 [2]的漸層網格的向量圖與對應網格，最左邊為美術人員手動產生，可以看到最接近原圖，中間為非美術人員初始化的結果，最右邊是中間的圖依顏色梯度最佳化 引用自 [2]

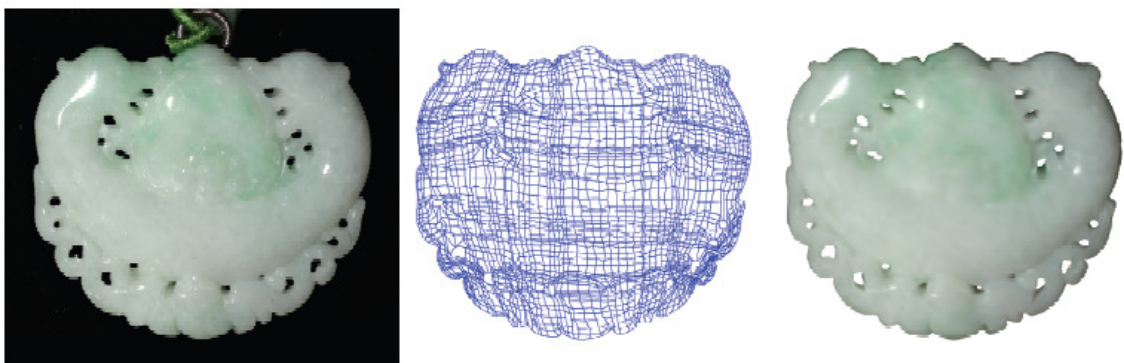


Figure 2.5: Lai等人 [3]的漸層網格的向量圖與對應網格，此向量圖使用340個控制點 引用自 [3]

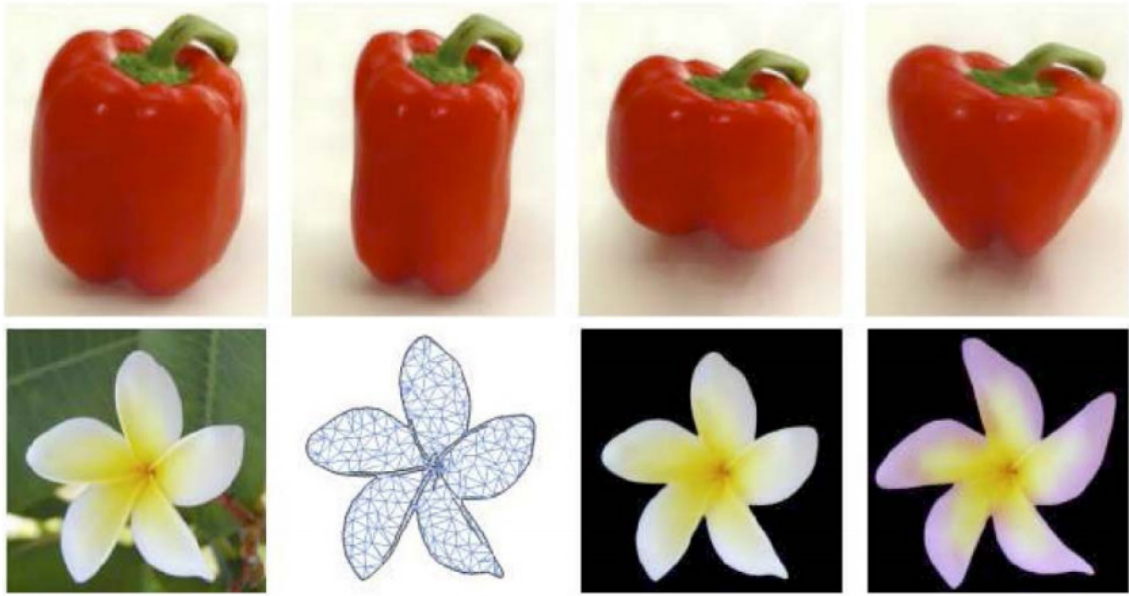


Figure 2.6: 上方為編輯變形後的向量化結果，下方為改變顏色的向量化結果引用自 [4]

使用平滑的三角面來呈現向量化結果，此方法先在顏色變化劇烈處產生特徵曲線，再對這些特徵曲線三角化，再根據原圖分群與細化。此論文利用拉伸三角化後的模型來達到編輯向量圖的功能，同時也提出改變特徵曲線顏色來得到不同的向量化結果，如圖2.6所示。

Selinger [5]提出了另一種名為Potrace的向量化方法，如圖2.7所示，這種方法能將點陣圖自動轉換成平滑的輪廓曲線，此方法需要明確的二值化圖片，若需將彩色影像做轉換，通常會將圖片轉換為灰階再做處理，再依不同亮度做處理，所計算出來的結果，會在漸層處產生大量區塊，如圖2.8所示，此方法非常依賴二值化的前置處理，而卡通動畫通常顏色多變，將大大增加前置處理的困難度。

與過去偵測線寬的方法比較 [9], [10], [11] 的偵測線寬方法比較，他們的方法都假設了線段的左右線寬是等寬的，如圖2.9(a)(b)(c)所示，而我們的方法如圖2.10所示，由於在彩色圖片中，筆觸的左右線寬常常不等長，因此我們尋找線寬的方法在彩色圖片中表現更好，我們的方法與ChenLMP15 [9]類似取亮度最低的點為線的中心，線的線寬則是根據亮度的梯度來取得，並使用了二次線性內

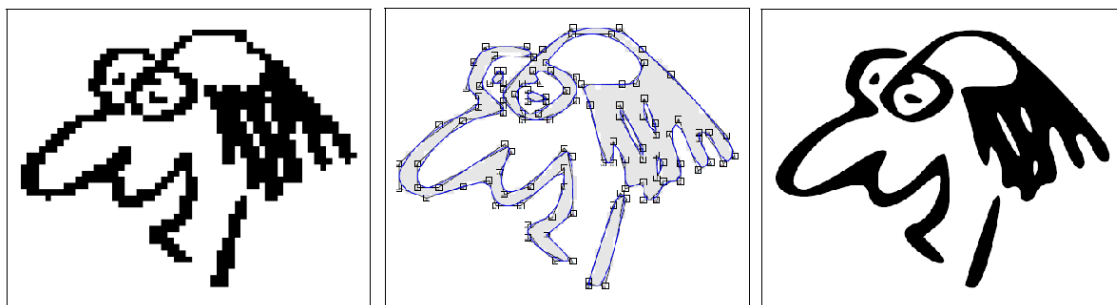
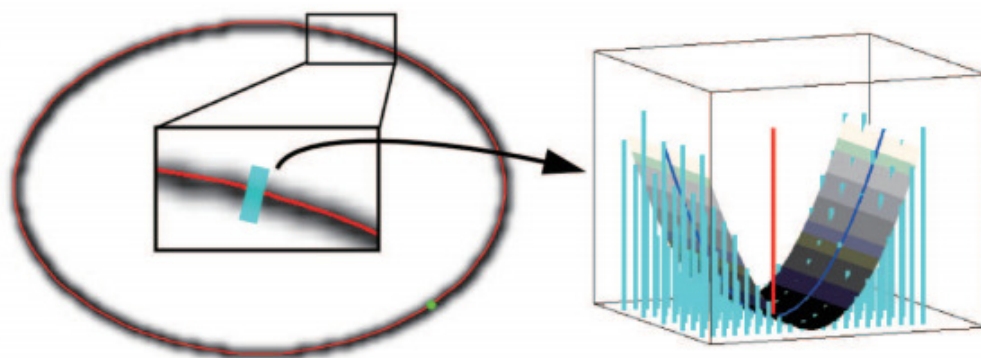


Figure 2.7: Selinger [5]的方法 引用自 [5]

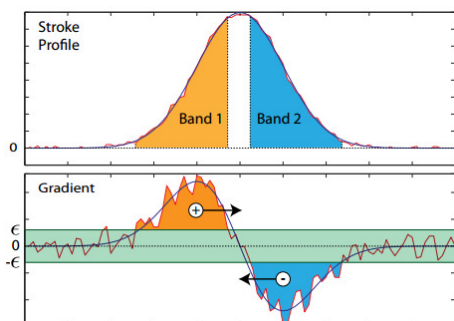


Figure 2.8: Selinger [5]使用灰階來分出顏色區塊

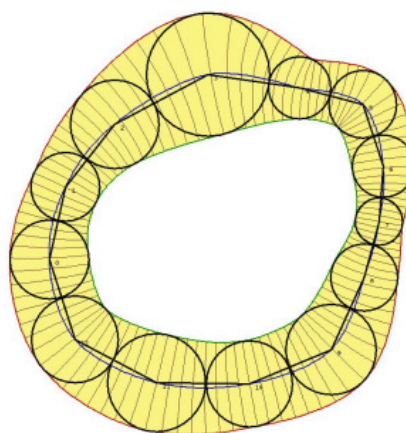
插來抓取非整數的線寬，與ChenLMP15 [9]先放大圖像後偏微分計算放大後的線條像素不同，另外他們在線寬只儲了一個值代表總線寬，我們的方法則分別儲存左右線寬，他們的方法在彩圖中遇到左右線寬不等長時會有線中心與線寬無法對應的問題，而我們偵測線寬的方法可以在彩圖上解決這個問題。



(a) 引用自ChenLMP15 [9]



(b) 引用自Noris13 [10]



(c) 引用自imta08 [11]

Figure 2.9: 過去找線寬與線中心的方法

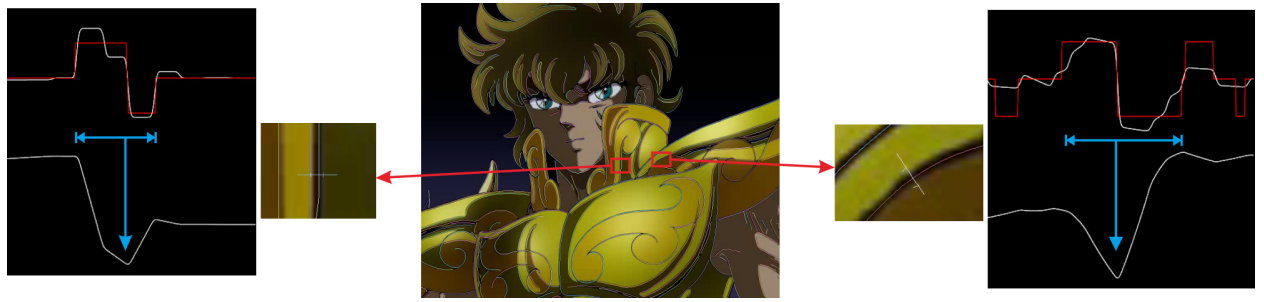
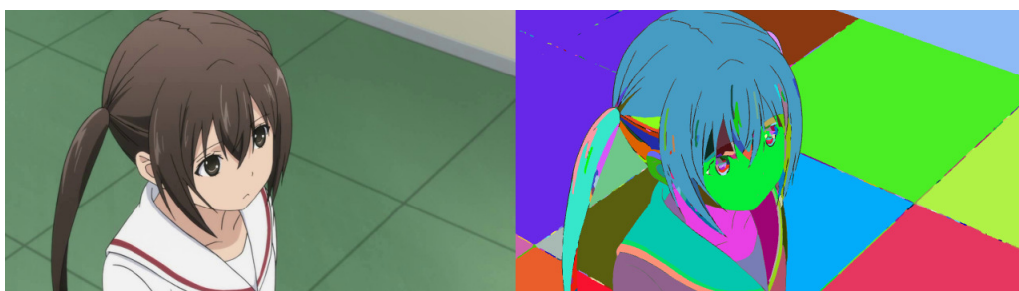
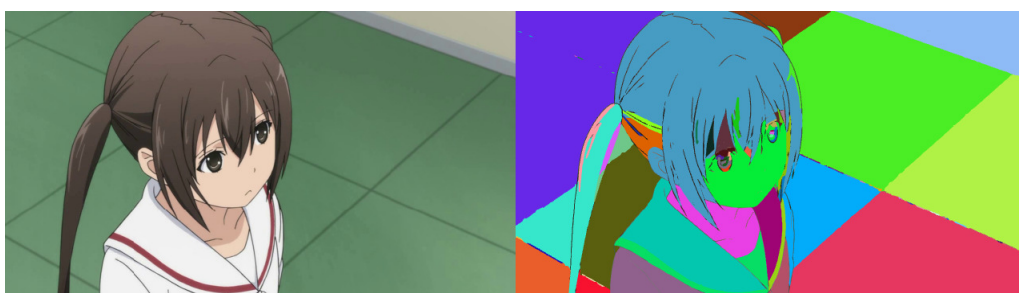


Figure 2.10: 我們的偵測線寬與線中心的方法

我們在每一影格向量化建立三角化後的顏色區塊後，比較前後影格最接近的三角形的分群，得到變動的區塊，再比較前後幀影像的顏色差距去除雜訊，得到一個能維持連續性的動畫向量化結果。相較於Song-Hai Zhang等人 [6]在維持連續性上一定要做背景重建，我們利用區塊三角化後的三角形對應，能在動態場景的不分前後景的維持住相似區塊的連續性，圖2.11為影格的對應圖，對應到的同一區塊會填上同樣的顏色，可以看到大部份的區塊都有對應到，沒有對應到的部份我們會將其視為新的區塊。



(a) 對應三角化區塊一影格1



(b) 對應三角化區塊二影格5

Figure 2.11: 影格對應示意圖，左圖為原圖，右圖為區塊顏色示意圖，對應到的同一區塊會填上同樣的顏色

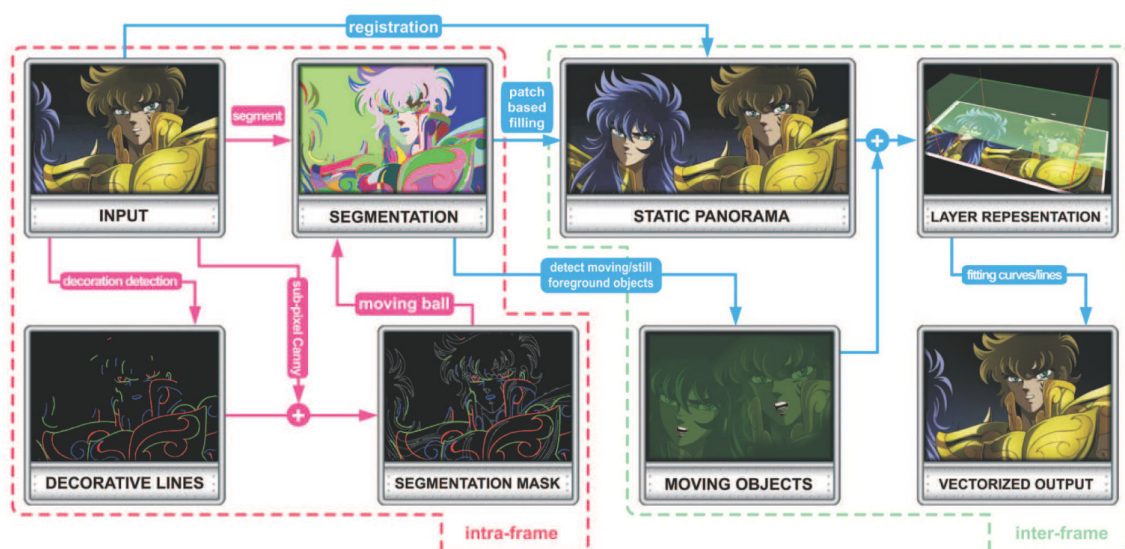


Figure 2.12: Vectorizing Cartoon Animations 流程圖 引用自Zhang09 [6]

2.2 動畫向量化

由於點陣圖有放大失真的問題，加上近年來螢幕的解析度逐漸增加，傳統的影片壓縮如果解析度在1080p時20分鐘的影片就需要1.8GB，當解析度在2160p時空間大約需要四倍，4k需要八倍大的空間，而向量圖因為檔案大小與解析度無關，加上動畫通常是以有漸層的色塊與線條組成，與真實影像大量的細節相比，更適合向量化。

若能將一般影片格式儲存的動畫轉換成向量化的動畫，將可適應更高解析度的顯示器，而點陣影片在較高解析度的顯示器中，會有鋸齒模糊的現象產生，希望本研究能透過將動畫向量化以解決高解析度時影片模糊的問題。

目前在動畫向量化最具代表的是由Song-Hai Zhang等人提出的Vectorizing Cartoon Animations [6]，裡面包含了動畫影像特徵的萃取，與向量化動畫化的方式，並分析影片的動態人物與靜態場景部份，減少影片的閃爍問題，且成果可輸出成flash可編輯的fla格式，可讓美術人物進行編輯與後製。

從圖2.12中可以看出此論文將向量化分為兩部份，單張影像向量化與動

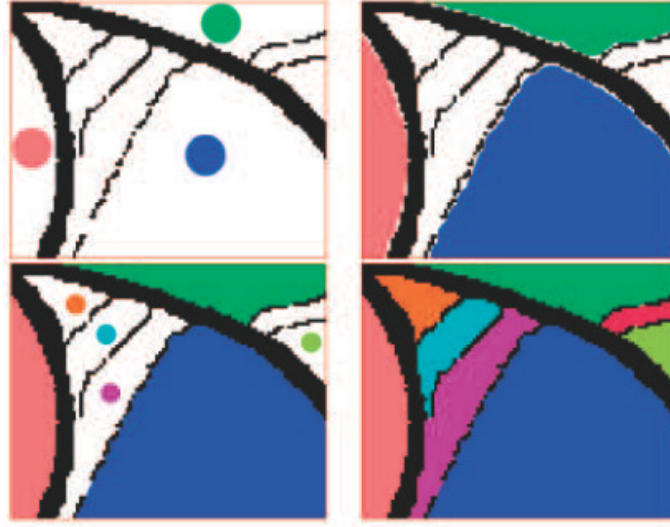


Figure 2.13: Trap-Ball Region Segmentation 流程示意圖 引用自Zhang09 [6]

畫向量化。單張的部份先從輸入圖片產生筆觸(Decorative Lines)與分割遮罩(Segmentation Mask)再用陷阱球(Trap-ball)把分割區塊(Segmentation)產生出來，再來是靜態場景部份，先使用SLIC [12]得到移動向量與旋轉方向，用平均值算法把背景計算出來，從背景與分割區域中計算出移動物件(Moving Objects)，最後將背景的向量化並貼上移動物件的向量化得到向量動畫。

此論文首先利用Canny [13]找到兩條相近平行的線，且中間有用Cheng09 [7]所偵測到的線，找到筆觸，此論文改善了Steger98 [14]的連線演算法來連接所有可能將筆觸連起來的點，假設這些點在線上的法向量為 $\dots, k-2, k-1, k$ ，把權重平均加在每一個點上，參數 ω 要在(0,1)區間，此論文在他們的範例中使用0.3當 ω 。

$$\alpha'_k = \sum_i \omega^{k-i} \alpha_i / \sum_i \omega^{k-i} \quad (2.1)$$

再對Canny [13]演算法所生成的線測出來的線條做陷阱球分割法，這個方法首先用大的圓當遮罩如圖2.13左上所示，把範圍內沒有白線的部份標記起來，用這方法對整張圖做一次，標記完會像圖2.13右上所示，再用區塊擴張(Region Growing)補齊邊界小區塊沒被陷阱球遮到的部份，再用更小的圓形遮罩去圈出更小的區塊如圖2.13右下所示，最後分割完的區域結果如圖2.14所示。



Figure 2.14: Trap-Ball Region Segmentation 結果示意圖 引用自Zhang09 [6]

分割完的區塊被當作一塊塊的色塊，這時還有細細的白線需要分區，要有較平滑的分區結果，白線最好是被左右兩邊其中一邊的區塊分走，因此白線會與兩邊相鄰的區塊做顏色距離的比較，並將白線填為顏色距離較近的區塊的顏色，可以參考圖2.15，若白線的顏色用每個點各自選顏色距離最近的區塊，有可能同一條白條分到兩邊不同的區塊會出現圖2.15(c)的結果。



(a)

(b)

(c)

Figure 2.15: 將白線補完示意圖，圖(a)中的白線，在原圖中的顏色比較靠近藍色的區域塊，因此全部填成藍色如圖(b)所示，沒有全部都分到同一顏色區塊可能會出現圖(c)的情況。

所有的區塊都被分割出來後，再用公式2.2對每個區塊做Color Modeling，為每個色塊建立Color Model，以此當作區塊顏色。

$$f_i(x, y) = a_{i0} + a_{i1}x + a_{i2}y + a_{i3}x^2 + a_{i4}xy + a_{i5}y^2 \quad (2.2)$$

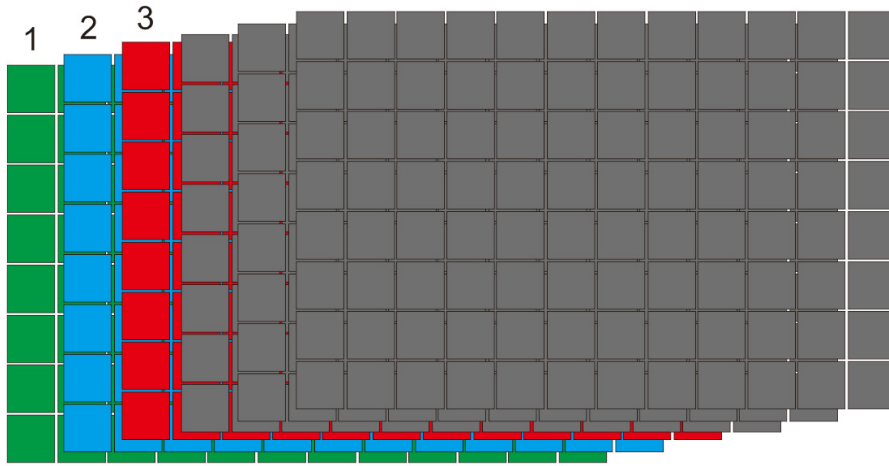
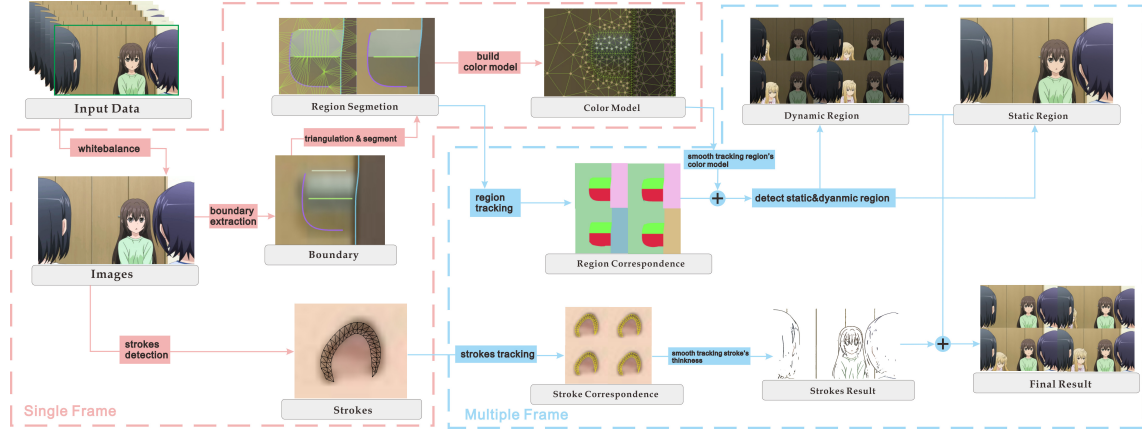


Figure 2.16: 影像重疊加總示意圖

在這篇論文中他們也分析了動態與靜態場景，並針對靜態場景做優化，讓靜態場景的部份更連續，減少Frame與Frame之間的分別轉換成向量圖後的差異，讓人眼比較不會感受到不連續。他們的方法分析靜態場景的方法是使用SIFT [12]把圖片的移動向量與縮放旋轉資訊計算出來再用平均值算法把背景計算出來。平均值算法就是將圖片重疊的部份加總起來取平均，以圖2.16為例，假設每一幀就右移一個像素，最左邊為第一行，所以第一行的平均值就是自己，第二行的平均值就是兩張圖的第二行各自的像素的平均值，第三行的平均值就是三張圖的第三行各自像素的平均值，以此類推，可以得到整張背景圖如圖2.12的Static Panorama所示。

而移動物件(Moveing Object)則是他們將分割完的顏色區塊，各別去測試在背景圖的變化差異，當該顏色區塊與背景圖的顏色距離太大時，他們將該顏色區塊辨識為移動物件，在影片向量化的時候，覆蓋在背景向量化上，完成他們的影片向量化。

Chapter 3 論文簡介



(a)

Figure 3.1: 總架構圖

本章簡單介紹，我們的動畫向量化流程，我們系統的輸入資料為2D點陣動畫影格，圖 3.1介紹了系統運作過程分成二個部分，一個是單一影格的處理，另一個是相鄰影格的處理。在單一影格處理中，首先使用卡通筆觸萃取 [7]的方法偵測筆觸，此方法主要利用深色筆觸會在原始影像經過二階微分處理後，會在深色筆觸的中心有較高的值的特性來得到顏色的筆觸的骨架，但在彩色影像中的結果往往會有些微偏差需要校正，因此我們依據該方法得到的筆觸骨架與原圖顏色梯度分析計算出線中心與線寬，在色塊分割的方法，再從影格中透過卡通筆觸萃取 [7]與坎尼運算子(Canny) [13]的混合方法偵測出邊界並透過德洛內演算法(Delaunay)[19]將邊界三角化，經由修補邊界後得到分割後的顏色區塊，這些色塊以三角化網格的方式儲存，區塊內的顏色模型是以三角形的頂點位置取樣建立，筆觸以樣條函數(spline)控制點的方式儲存位置與左右線寬。為了維持連續性，我們對接著對相鄰的多個影格所偵測的筆觸及色塊做對應，對應的方法是利用兩張相鄰影格中區塊的每塊三角形的中心去做區塊上的對應利用兩張相鄰影格中區塊的每塊三角形的中心去做區塊上的對應，這些對應的三角最再利用投票機制得到每個區塊的對應區塊，攝影機移動所造成的偏移使用縮放不變特徵轉

換(Scale-invariant feature transform)[12]來計算，筆觸部份也是使用位置資訊與投票的方式來做相鄰影格的對應，有了這些對應能讓我們分析出動態區塊與靜態區塊，並分析靜態區塊與影何來達到背景重建，將重建的影像當作關鍵影格，以更進一步加強時續連續性，並找到每一影格中所偵測到的所沒對應到的新筆觸及色塊，與有對應之筆觸及色塊，我們透過三角化對應將大小形狀接近的區塊維持在前一影格的狀態，筆觸部份則是平滑相鄰影格間的筆觸粗細，以維持其筆觸連續性以減少動畫的線條閃爍。有了先前步驟產生之色塊向量化與筆觸向量化，我們先將筆觸疊在色塊，再把動態區塊疊在靜態區塊，得到我們最終的動畫向量化結果。

Chapter 4 研究方法

在上一章我們簡介了我們的研究方法與流程，在這一章，我們會詳細講解每一步的演算法細節，包括單張影像向量化與動畫向量化的部份。

4.1 影像預處理

由於影片的影像一定有壓縮過，而依據壓縮的參數不同，會有不同程度的雜訊，而影片對比度與亮度更會直接影響影像特徵萃取的結果，導致後面的方法做出的結果不如預期，因此本論文先使用 L_0 Smoothing 將大部份的雜訊去除，再使用自動白平衡演算法，將太暗對比太低的影像調整成適合影像特徵萃取的狀態，最後再將影像做銳化，做銳化能將不明顯的邊界強化，讓之後的 Curve Extraction 找到的線更完整。

白平衡的演算法參考，我們參考Ching-Chih Weng的白平衡演算法 [8]，這個演算法的方法分兩部份，一是尋找白色參考點，二是依據白色參考點對原圖作調整。

尋找白點的部份：

1. 把圖的色彩空間從 RGB 轉換到 YC_bC_r
2. 將原圖切成12區塊
3. 計算每個區塊的 C_b, C_r 跟平均值 M_b, M_r
4. 算出 C_b, C_r 與 M_b, M_r 差值的平均 D_b, D_r

$$D_b = \sum_{i,j} (|C_b(i,j) - M_b|) / N$$
$$D_r = \sum_{i,j} (|C_r(i,j) - M_r|) / N$$

5. 如果該區的 D_b, D_r 的值偏小，代示該區塊顏色較平滑，就將這區塊的值忽略，最後平均剩下區塊的 D_b, D_r, M_b, M_r 當作整張圖的 D_b, D_r, M_b, M_r
6. 將符合下列公式標為可能白色參考點

$$|C_b(i, j) - (M_b + D_b \times \text{sign}(M_b))| < 1.5 \times D_b$$

$$|C_r(i, j) - (1.5 \times M_r + D_r \times \text{sign}(M_r))| < 1.5 \times D_r$$

7. 對可能的白色參考點，取最亮的10%為最終確定的白色參考點

依白點調整影像的部份：

1. 計算白點的平均值，RGB三個通道分開計算 $R_{avew}, G_{avew}, B_{avew}$
2. 算出通道增益， Y_{max} 是 YC_bC_r 的Y

$$R_{gain} = Y_{max} / R_{avew}$$

$$G_{gain} = Y_{max} / G_{avew}$$

$$B_{gain} = Y_{max} / B_{avew}$$

3. 調整影像

$$R' = R \times R_{gain}$$

$$G' = G \times G_{gain}$$

$$B' = B \times B_{gain}$$

從圖4.1可以看到未白平衡的圖片，在抓取衣服的顏色邊界時抓不到，且桌角的邊界也遺失了一小段，但是做了白平衡可以解決這個問題。



(a) 原圖



(b) 未白平衡的向量化



(c) 白平衡後的向量化

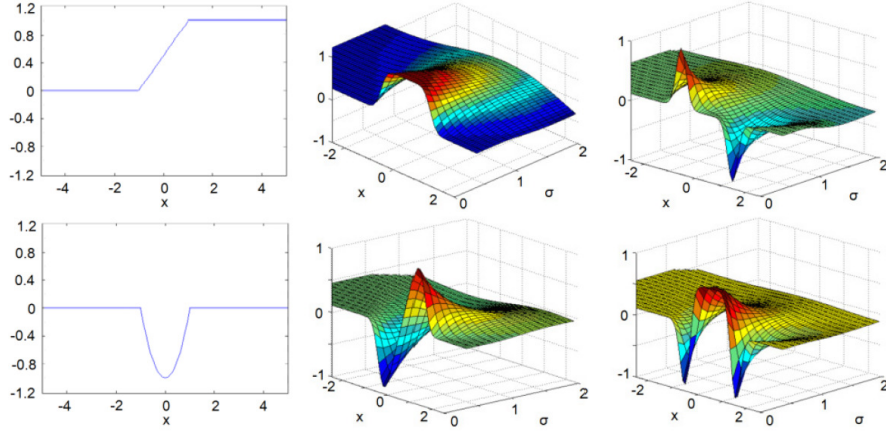


Figure 4.1: 圖形切面與訊號微分的示意圖，第一列為顏色邊界的視覺化，第二列為裝飾曲線的視覺化，第一行是切面亮度，第二行為一階微分，第三行為二階微分 引自Cheng09 [7]

4.2 動畫特徵截取

Ming-Ming Cheng在2009年提出的Cheng09 [7]為目前效果最好的演算法，圖4.2第一列為顏色邊界(Boundary curves)的切面圖，左下角圖為裝飾曲線(Decorative Curves)切面圖，他們的裝飾曲線即本論文的筆觸，中間兩張圖為微分一次的結果，右邊兩張圖為微分兩次的結果，此圖說明了顏色邊界(Boundary Curves)可以透過一次微分後偵測較大的值來尋找，而筆觸以透過二次微分後偵測較大的值來尋找，此方法中描述了如何的二次微分的方式來找到筆觸，但在顏色邊界的部份切出來的線會比較偏顏色較暗的一邊。

$$f_b = \begin{cases} 0 & x < -\omega_b \\ \frac{h}{2} + \frac{h}{2\omega_b} & -\omega_b \leq x \leq \omega_b \\ h & x > \omega_b \end{cases} \quad (4.1)$$

$$f_d = \begin{cases} ax^2 - h & |x| \leq \omega_d \\ 0 & |x| > \omega_d \end{cases} \quad (4.2)$$

本論文為定義了兩種線條的函數，公式4.2示範了一維空間下的例子，顏色邊界的輪廓為 f_b ，如圖4.2的第一行第一列，筆觸的輪廓為 f_d ，如圖4.2的第一行第二列， W 為線寬， h 為線寬極大值，在大部份的動畫裡的情況中 W_d 大於 W_b 。

4.2.1 顏色邊界萃取

本方法將以Curve Structure Extraction for Cartoon Images為基礎做改善，原始的方法找到的結果如圖4.2所示，在黑線(decorative curves)偵測的方式表現良好，但在顏色邊界(boundary curves)的部份切出來的線會比較偏顏色較暗的一邊，而且邊界與線段之間，線段沒有連接起來，本研究改善了這兩部份。

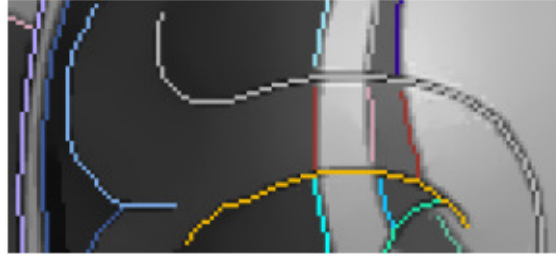


Figure 4.2: Curve Structure Extraction for Cartoon Images 引用自Cheng09 [7]

為了解決 f_d 在顏色邊界(boundary curves)偏顏色較暗的一邊的問題，我們在 f_{d2} 裡混合了 f_{v1} 與 f_{v2} 的權重，並使用 λ 來控制兩邊的比重， f_{v1} 所代表的意思是在遇到顏色邊界(Boundary Curves)如圖4.2中上微分一次時值較大的部份與右上微分兩次時的絕對值成反比，而黑線(Decorative Curves)在圖4.2中下微分一次與圖4.2右下微分兩次時的比較也可以發現，微分兩次值高的部份與微分一次的絕對值成反比，藉由以上兩種觀察，我們可以定義出 f_{v1} 與 f_{v2} 的權重公式如公式4.3,4.4，並用 λ 來控制兩者的比重如公式4.5，最後我們可以得到包含了黑線(Decorative Curves)與顏色邊界(Boundary Curves)的Curves集合。

$$f_{v1}(x) = \begin{cases} f'(x) * (1 - \text{abs}(f''(x))) & f'(x) > 0 \\ 0 & \text{else} \end{cases} \quad (4.3)$$

$$f_{v2}(x) = \begin{cases} f''(x) * (1 - \text{abs}(f'(x))) & f''(x) > 0 \\ 0 & \text{else} \end{cases} \quad (4.4)$$

$$f_{d2}(x) = \lambda * f_{v1}(x) + (1 - \lambda) * f_{v2}(x) \quad (4.5)$$

但在這步得到的線到需要經過處理才能使用，原因是這一步抓到的線條沒有辦法明確的分隔顏色區塊，我們基於線段若跟端點夠接近應該連接在一起的想法分析，這些線段在原圖上應該是要能把顏色區塊分隔開的，因此我們要將線段端點與最近的邊界、端點連接起來，我們分析發現了四種端點與邊界的情況，分別是，一個端點與一個邊線如圖4，兩個端點與一個邊線如圖1, 3，兩個端點沒有邊線如圖2，其它更複雜的情況都可以使用距離近的優先處理的方式，簡化成這四種情況，下面說明本論文使用了三角化，並檢查所有線段頂點的鄰近節點來連接可以圍成顏色區塊的線段，分析了四種邊界曲線相連的情況，並使用在我們的區塊分群中。

接下來介紹四種情況，示意圖中的紅線與藍線是曲線萃取完的邊界曲線，黑線為其它邊界曲線，綠色的線是三角化後從紅線與藍線長出的線段，紅線與藍線將會在綠線中選一段來將彼此連結起來，下面介紹三角化後會遇到的四種情況。

1. 兩個頂點都連接到同一個邊界節點時優先連接 如果有超過一個以上的邊界，就連最近點。

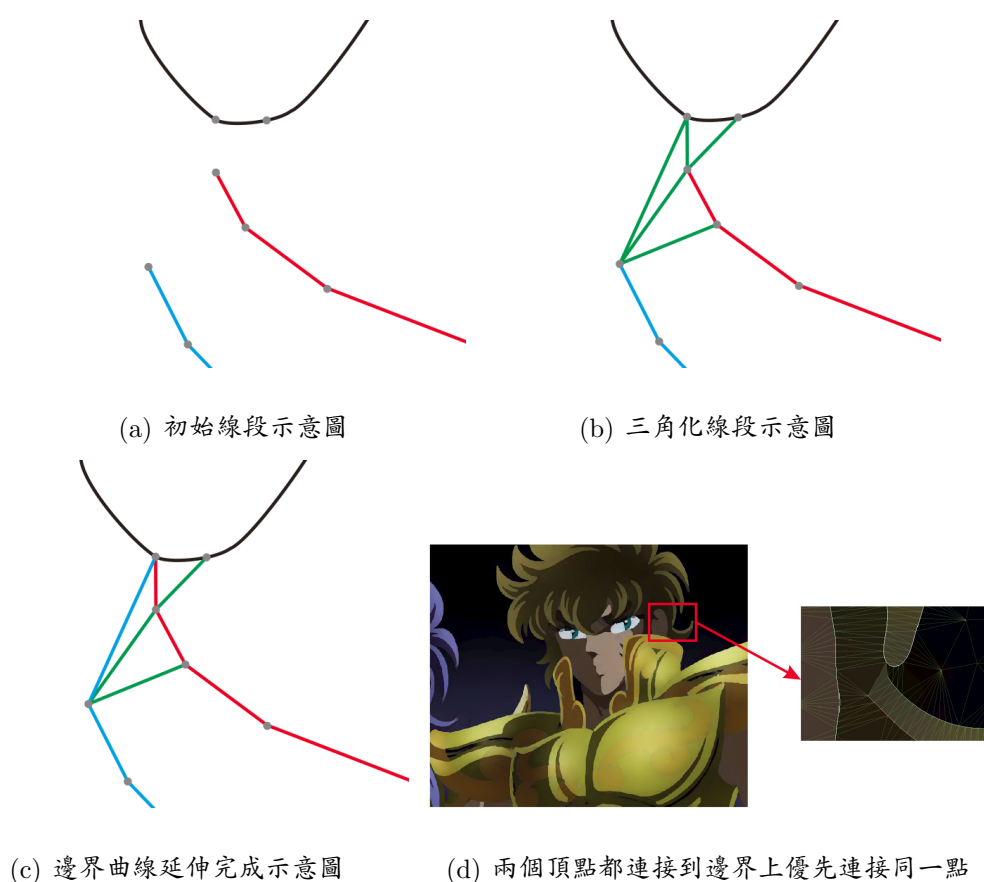


Figure 4.3: 兩個頂點都連接到同一個邊界節點時優先連接，紅線與藍線是要連接的邊界曲線，綠線是頂點三角化後有跟邊界連接的線，邊界曲線延伸完成示意圖上的紅線跟藍線是連接完成的邊界曲線，最後是真實遇到的情況示意圖

2. 兩個頂點可以相連時連結在一起

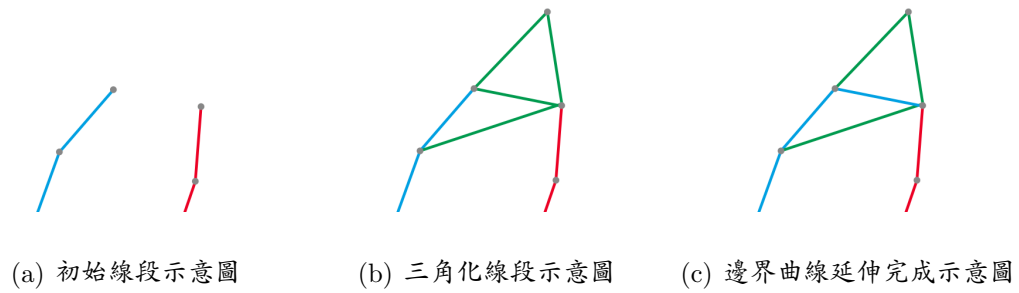
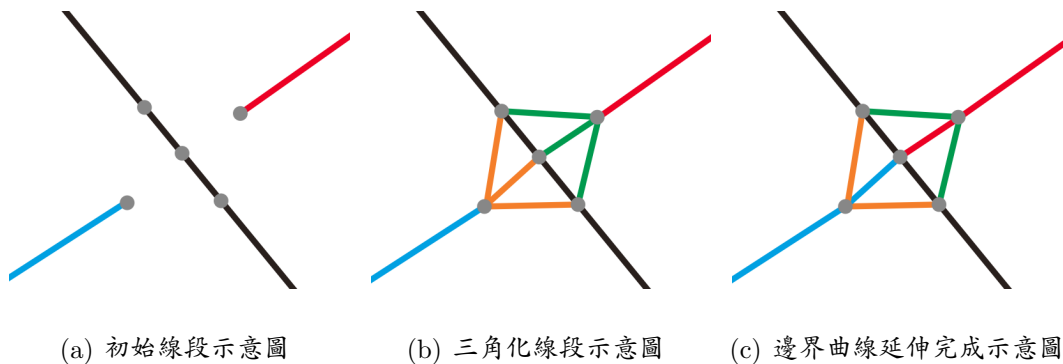


Figure 4.4: 兩個頂點可以相連時連結在一起，紅線與藍線是要連接的邊界曲線，綠線是頂點三角化後有跟邊界連接的線，邊界曲線延伸完成示意圖上的紅線跟藍線是連接完成的邊界曲線，最後是真實遇到的情況示意圖

3. 兩個頂點可以交成十字型時優先連接在同一點 連結時優先連結連平行原線段的線段。



(d) 兩個頂點可以交成十字型時優先連接在同一點

Figure 4.5: 兩個頂點可以交成十字型時優先連接在同一點，紅線與藍線是要連接的邊界曲線，綠線是頂點三角化後有跟邊界連接的線，邊界曲線延伸完成示意圖上的紅線跟藍線是連接完成的邊界曲線，最後是真實遇到的情況示意圖

4. 選擇最平行線段的邊界節點連接

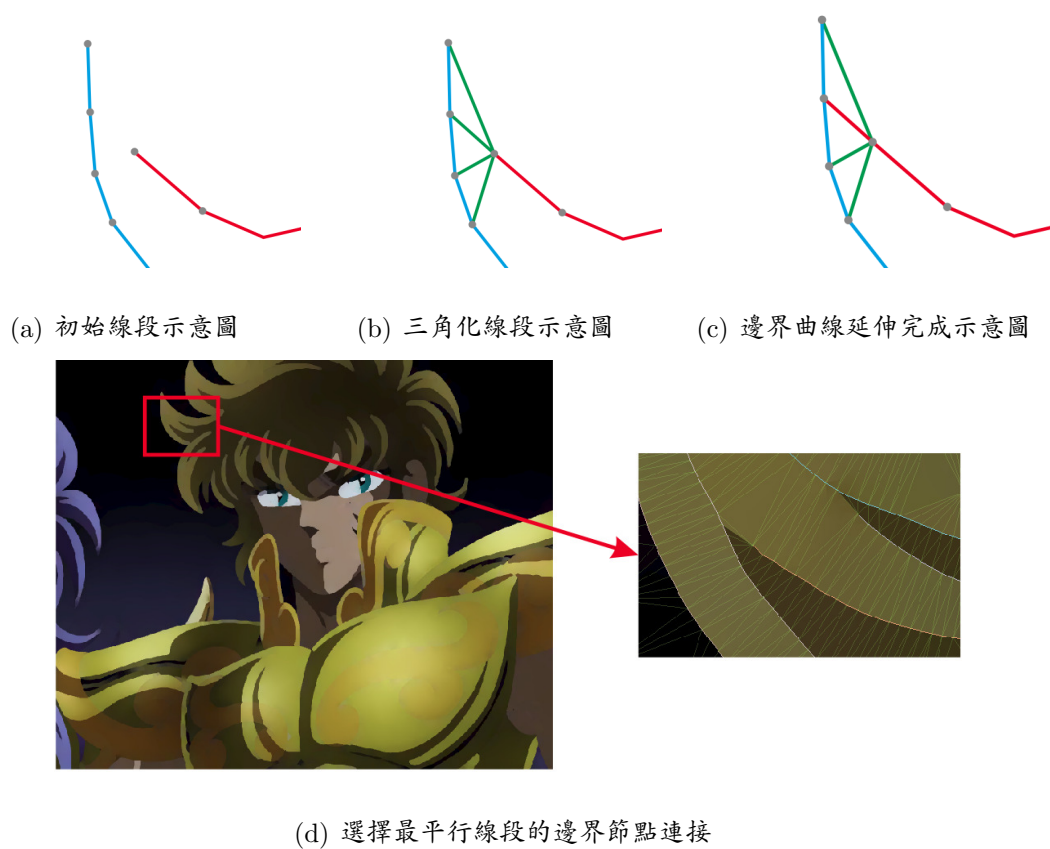


Figure 4.6: 選擇最平行線段的邊界節點連接，紅線與藍線是要連接的邊界曲線，綠線是頂點三角化後有跟邊界連接的線，邊界曲線延伸完成示意圖上的紅線跟藍線是連接完成的邊界曲線，最後是真實遇到的情況示意圖

4.3 顏色特徵萃取

顏色模型直接影響了向量化的結果，我們實驗了多項式模型與RBF模型，多項式顏色模型無法滿足複雜的顏色情況，且會有少部份數值誤差解出錯誤顏色的問題，一般的RBF模型在生成與計算花費過多的時間，因此本論文使用了以節點為基礎的顏色內插模型，在求值時只尋找離座標最近的鄰近節點來做顏色內插，我們的顏色模型如圖4.7所示，是一個網格的結構，網格避開了筆觸與顏色邊界的部份，從原圖取樣得到顏色，取樣的顏色存在每個網格節點中，在需要內插時，將內插點鄰近的所有節點顏色線性內插得到顏色。

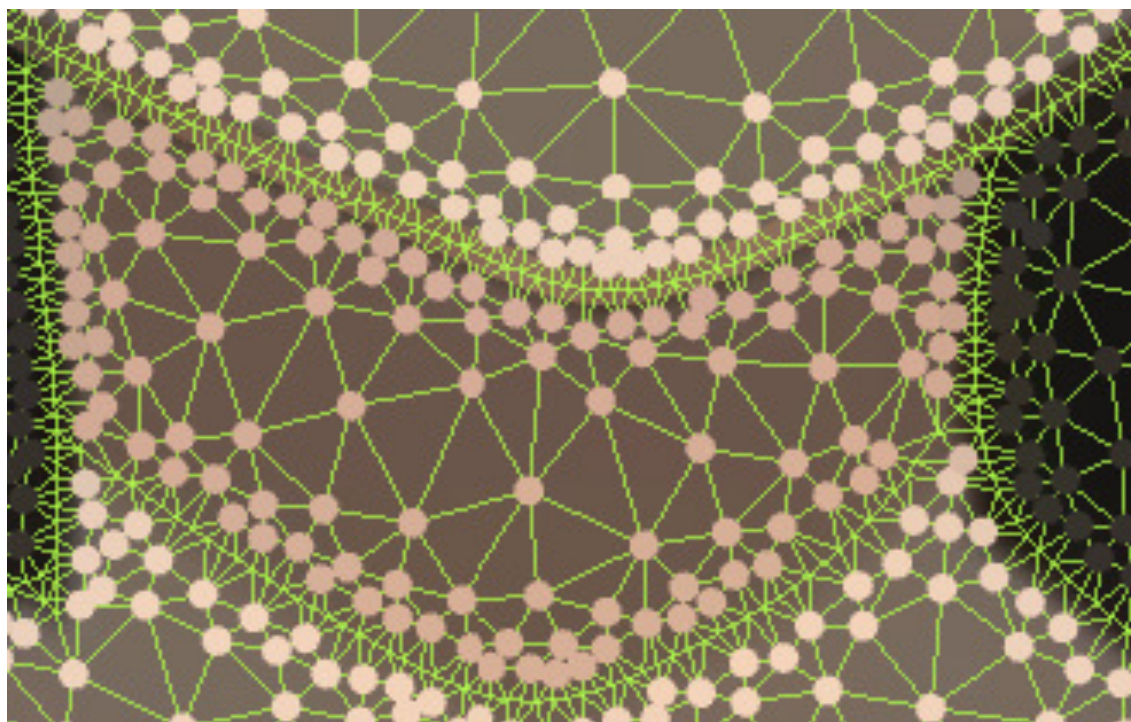


Figure 4.7: 顏色模型取色視覺化

4.3.1 筆觸萃取

從上個步驟中得到顏色邊界後，在這個步驟要將筆觸分離出來，我們將測到的線的法向量在原圖上做亮度取樣如圖4.8(a)的部份，基本概念是設定斜率的門檻來

得到線寬，從垂直於筆觸的位置取出亮度值，會呈現出一個山谷如圖4.8b所示，根據亮度得到的斜率我們發現，山谷會在中間，從中間往左的斜率會是正值，往右的斜率會是負值，離開筆觸後斜率會接近0，以此為基準，我們找到判斷筆觸的方式，設斜率為 S ， S_n 為中心位置， S_m 為最右邊位置，線左半邊的sample點亮度為 L ，右半邊為 R ，並將判斷筆觸的演算法寫在演算法1。



(a) 在Constraint Curves上做顏色取樣 (b) 取樣後的亮度與斜率視覺化;下方白的視覺化;圖中的綠線為為Constraint 線為取樣的亮度視覺化，紅線表示超過Curves，垂直的白線為取樣的位置，在 門檻值後二值化的結果，在本論文中門白線上平行綠線的兩條小短線為量測到 檻值為15度角。
的線寬。

Figure 4.8: 筆觸的梯度變化

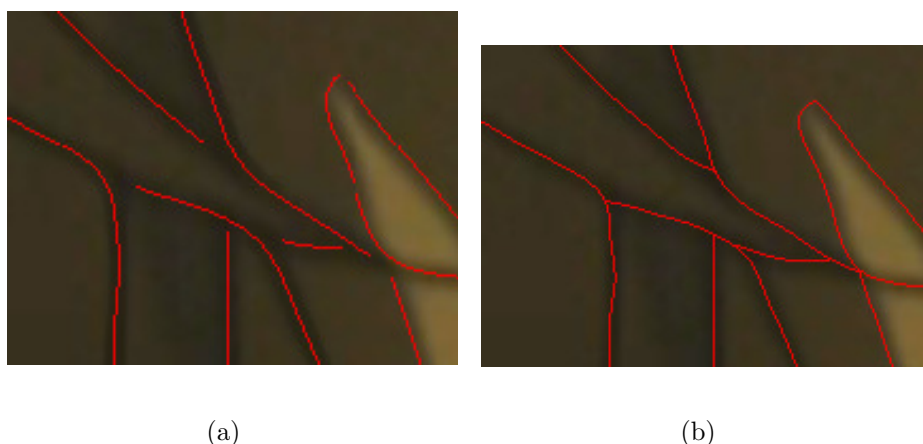


Figure 4.9: 斷裂的線，(a)為修補前，(b)為修補後

由於上一步找到的Decorative Line有常會有斷裂的問題，所以還需要修補，目

前我們只會簡單的延長線段的兩邊，在短距離內如果能找到其它線段，就將線段連起來，如圖4.9所示。

4.3.2 筆觸向量化

在取得了筆觸的資訊後，爲了要能讓以使用者覺得舒適的方式視覺化出來，可以調整線的漸層或明暗，圖4.10(a)爲普通的筆觸(Decorative Curves)，右邊爲有淡出效果的筆觸，通常沒有淡出效果能給人一種影像銳利的感受，因此本論文的實驗結果都使用普通的筆觸。

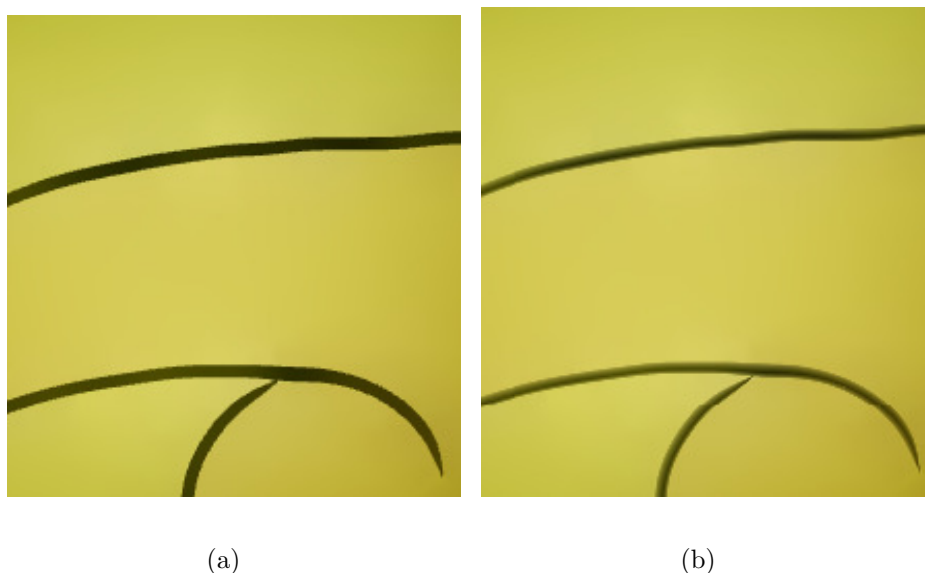


Figure 4.10: 筆觸向量化，(a)爲普通的筆觸，(b)爲淡出筆觸

斷裂的夾角線，如圖4.9所示，將其連接起來，若距離夠近，又角度接近則連接起來。。

Input: $L \leftarrow S_0 \dots S_n, R \leftarrow S_n \dots S_m$

Output: *boolean*

```
1 IsBlackLine = false; Lok = false; Rok = false;
2 for  $s \in L$  do
3   if IsBlackLine = false &  $s < 0$  then
4     return false;
5   end
6   if  $s > 0$  then
7     IsBlackLine = true;
8   end
9   if IsBlackLine = true &  $s \geq 0$  then
10    Lok = true;
11  end
12 end
13 IsBlackLine = false;
14 for  $s \in R$  do
15   if IsBlackLine = false &  $s > 0$  then
16     return false;
17   end
18   if  $s < 0$  then
19     IsBlackLine = true;
20   end
21   if IsBlackLine = true &  $s \geq 0$  then
22     Rok = true;
23   end
24 end
25 return Lok & Rok;
```

Algorithm 1: CheckIsBlackLinePoint

4.4 計算移動距陣

首先使用SIFT尋找移動距陣，SIFT可以找到影片中frame之間的關鍵點對應，如圖4.11所示，有了這些關鍵點的對應位置，再用中值法找到背景，但發現在橫向移動中，難免會有雜訊跟誤差，累積起來得到的背景就會有問題，在這裡還有一個問題，圖片向量計算出來往往不是整數而是浮點數，這個問題我們使用雙線性內插來實作有小數點距離的图片移動。

由於SIFT計算的移動向量配合中值法的結果不理想，從圖4.12，可以看到左邊人物的嘴角有破圖現象，使用中值法或平均法並不能得到可用的背景影像。

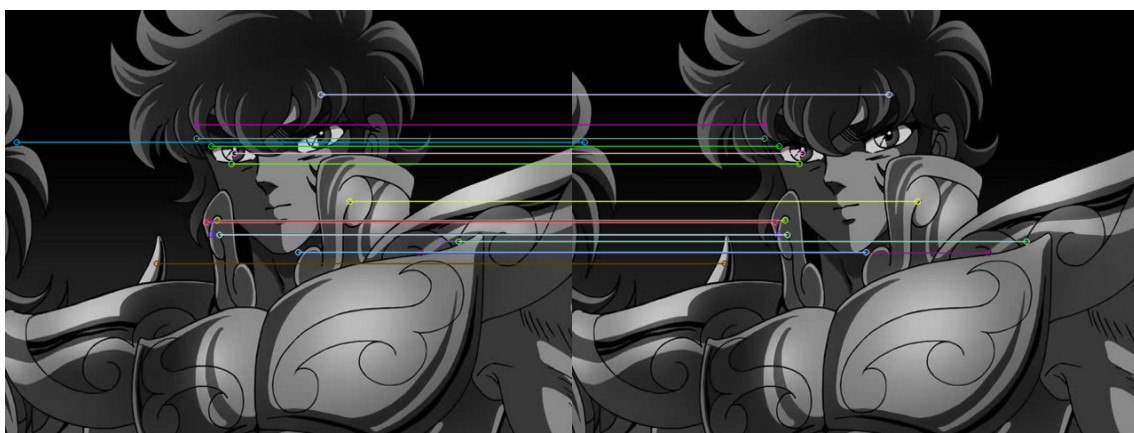


Figure 4.11: 使用SIFT計算的出來的對應點，可以用來計算移動向量

在融接圖片的演算法上，要減少這些誤差，必須在連續動畫中挑出相減後最小誤差的下一影格來做融接可以減少太多圖時產生的累積誤差，我們提出了一個演算法來融接圖片，該演算法在連續的影片圖像中找尋誤差最低的兩張做拼接，如演算法2所示， I 表示影片的連續影像 n 張， V 為影像之間的移動向量，演算法3計算兩張圖都有顏色的部份的距離，演算法 $ImgFillBlack4$ 把新的圖合進背景。

最後我們選擇SIFT當作我們計算移動向量的演算法，主要原因是速度快，計算出的結果比OpticalFlow還精確。



(a) 中值法 左邊的人嘴巴有錯誤



(b) 全部平均



(c) 我們的方法

Figure 4.12: 使用SIFT計算的向量，配合三種拼接演算法的結果

Input: $I \leftarrow img_0 \dots img_n, V \leftarrow v_0 \dots v_n$

Output: $BGImage$

```
1 for  $i \in I$  do
2   |   move  $img_i$  by  $V_i$ ;
3 end
4  $BGImage = I_0$ ;
5 for  $i \leftarrow 1$  to  $n$  do
6   |    $MinmumDistance = ColorDistance(I_i, BGImage)$ ;
7   |    $idx = i + 1$ ;
8   |   for  $j \leftarrow j + 1$  to  $j + 10$  do
9     |    $dis = ColorDistance(I_j, BGImage)$ ;
10    |   if  $dis < MinmumDistance$  then
11      |   |    $MinmumDistance = dis$ ;
12      |   |    $idx = j$ ;
13    |   end
14  |   end
15  |    $ImgFillBlack(I_{idx}, BGImage)$ ;
16 end
17  $ImgFillBlack(I_n, BGImage)$ ;
18 return  $BGImage$ ;
```

Algorithm 2: image stitching

Input: img_1, img_2

Output: $avgsum$

```
1  $sum = 0, count = 0;$ 
2 for  $p1_i \in img_1, p2_i \in img_2$  do
3   if  $p1_i$  has color and  $p2_i$  has color then
4      $count = count + 1;$ 
5      $sum = sum + abs(p1_i - p2_i)$ 
6   end
7 end
8  $avgsum = sum / count;$ 
9 return  $avgsum;$ 
```

Algorithm 3: ColorDistance

Input: img_1, img_2

Output: img_2

```
1  $sum = 0, count = 0;$ 
2 for  $p1_i \in img_1, p2_i \in img_2$  do
3   if  $p1_i$  has color and  $p2_i$  has color then
4      $p2_i = (p1_i + p2_i) * 0.5;$ 
5   end
6   else if  $p1_i$  has color and  $p2_i$  didn't has color then
7      $p2_i = p1_i;$ 
8   end
9 end
10 return  $img_2;$ 
```

Algorithm 4: ImgFillBlack

4.5 前景區塊萃取

在上一小節取得了背景後，就要處理動態的部份，我們先將每影格的影像與前後影格的影像做相減並加總起來如圖4.13，再取二值化如圖4.14將有變動的部份單獨向量化再貼回原向量化結果，使得靜態背景不會因為合成到顏色距離相差太大的圖片而模糊。在影格中的顏色區塊與關鍵影格的顏色區塊無法對應且該區塊前後影格的影像，顏色差距過大時，我們將該顏色區塊定義為動態區塊。



Figure 4.13: 有改變的部份的weight



Figure 4.14: 變動區域示意圖，有變動部份的區域用紅色標記起來

4.6 動態場景之區塊對應

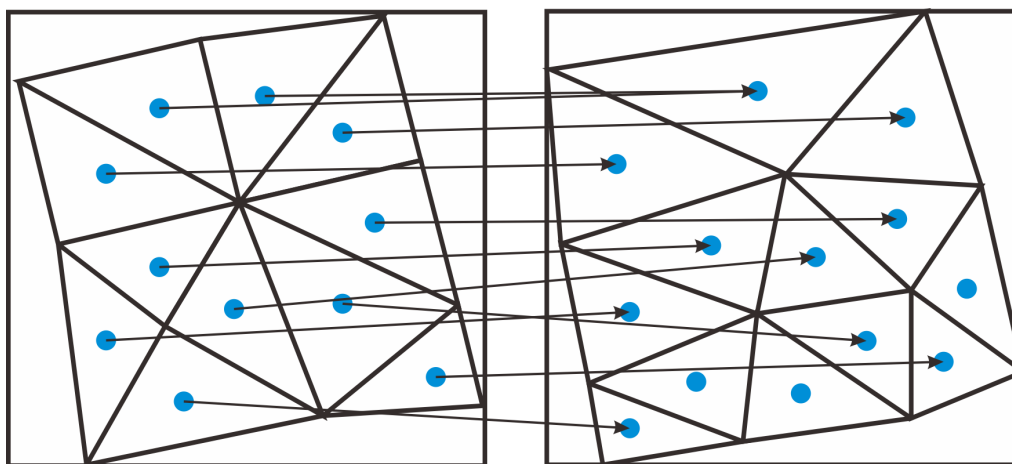
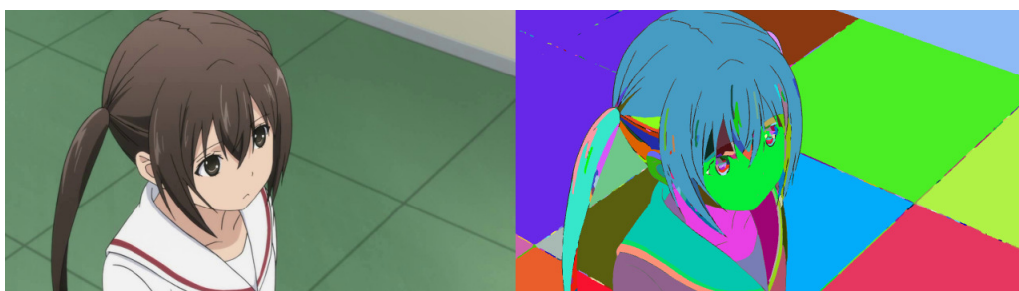
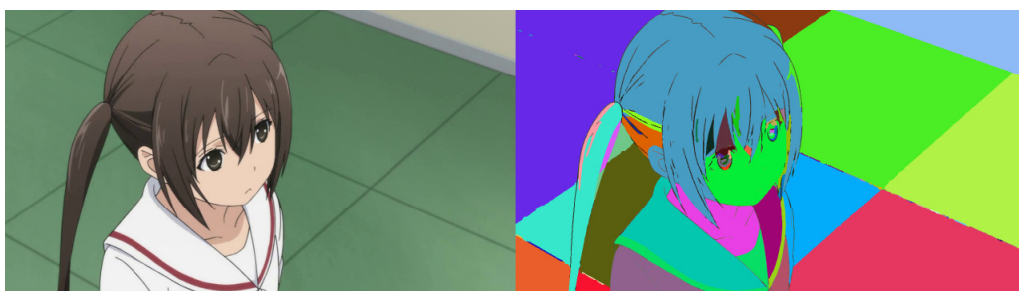


Figure 4.15: 區塊對應示意圖，圖中左邊的三角化要對應到右邊的三角化網格，藍色的點是每個三角形的重心

我們在做這一步之前先有了SIFT計算的位移矩陣，對應區塊的方法，如圖4.15所示，先將所有的區塊三角形，找到前一影格對應的三角形，為尋找前後影格中三角形重心最接近的兩三角形做區塊對應，這個對應座標在比較前必須要先乘上SIFT的位移矩陣，最後用投票的方式，統計該區塊的三角形對應到哪個區塊最多，來決定每個區塊的對應，對應不起來的區塊則視為新區塊，在我們的論文中三角化後的三角形數量相差十分之一以上的，視為無法對應的區塊，最後可以從圖4.16看到動態對應的結果，我們會將對應的區塊比較大小接近，前後影格後顏色近似的區塊不變。



(a) 對應三角化區塊一影格1



(b) 對應三角化區塊二影格5

Figure 4.16: 影格對應示意圖，左圖為原圖，右圖為區塊顏色示意圖，對應到的同一區塊會填上同樣的顏色

Chapter 5 實驗結果

本實驗中利用六張影像與一部影片中的第一張影像做為實驗的測試圖，圖5.1, 5.9解析度為640x480，圖5.7, 5.8, 5.10, 5.12解析度為1280x720。我們將利用前兩張測試圖，與Zhang2009 [6]做比較，並附上我們其它圖片的結果，由於在每張圖中我們的參數不同，下面的結果會在圖後面加註向量化參數，筆觸的參數有mask size, threshold 1, threshold 2，我們使用Sms, St1, St2代表，邊界的參數mask size, threshold 1, threshold 2，我們使用Bms, Bt1, Bt2表示。

與Zhang等人 [6]相比，以區塊為基底的向量化筆觸會有些微粗細不一，但我們分開處理的筆觸不會有這個問題，我們能藉由調整參數處理在原圖上較模糊、對比低、顏色暗的筆觸與邊界。



(a) 原圖



(b) 向量圖



(c) 引用Zhang09 [6]

Figure 5.1: 結果一($Sms=5$, $St1=0.9$, $St2=0.05$, $Bms=5$, $Bt1=1.6$, $Bt2=0.05$)



Figure 5.2: 左為他們的向量化中間為原圖右邊為我們的向量化，以區塊為基底的向量化筆觸會有些微粗細不一，但我們分開處理的筆觸不會有這個問題

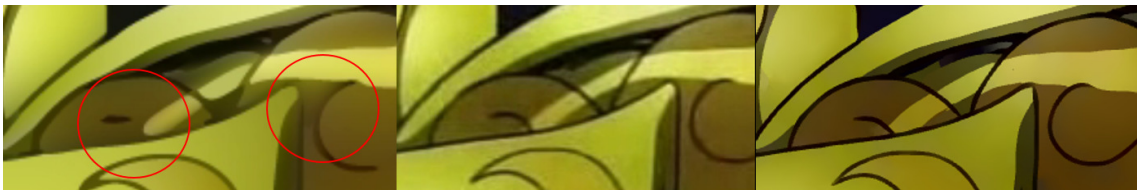


Figure 5.3: 左為他們的向量化中間為原圖右邊為我們的向量化，他們在原圖有筆觸的地方但他們沒萃取到的部份，還會有因為筆觸沒有被萃取造成色塊顏色被弄髒的問題，但在我們的結果中都能忠實的呈現

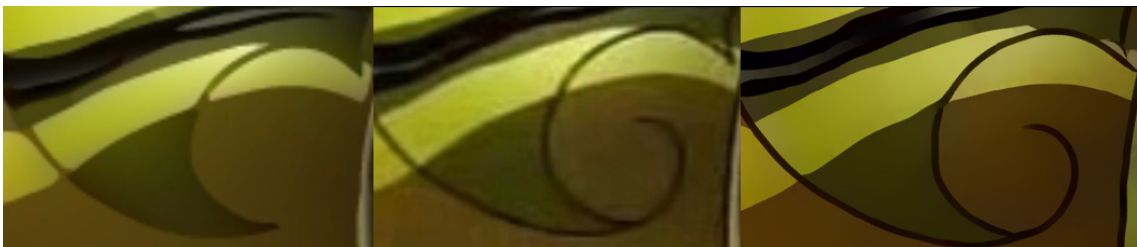


Figure 5.4: 左為他們的向量化中間為原圖右邊為我們的向量化，他們在原圖有筆觸的地方但他們沒萃取到的部份，還會有因為筆觸沒有被萃取造成色塊顏色被弄髒的問題，但在我們的結果中都能忠實的呈現



Figure 5.5: 左為他們的動畫向量化中間為原圖右邊為我們的向量化，頭髮的因為有了筆觸而有更豐富的細節，因為我們能處理顏色暗對比低的筆觸



Figure 5.6: 左為他們的動畫向量化中間為原圖右邊為我們的向量化，圖左他們的頭髮與背景融合了，但我們的偵測能分成兩個邊界得到較好的向量化結果



(a) 原圖



(b) 向量圖

Figure 5.7: 結果二($Sms=5$, $St1=0.9$, $St2=0.05$, $Bms=5$, $Bt1=0.3$, $Bt2=0.05$)



(a) 原圖



(b) 向量圖

Figure 5.8: 結果三($Sms=5$, $St1=0.9$, $St2=0.05$, $Bms=5$, $Bt1=0.9$, $Bt2=0.05$)



(a) 原圖



(b) 向量圖

Figure 5.9: 結果四($Sms=5$, $St1=0.9$, $St2=0.05$, $Bms=5$, $Bt1=1.6$, $Bt2=0.05$)



(a) 原圖



(b) 向量圖

Figure 5.10: 結果五($Sms=5$, $St1=0.9$, $St2=0.05$, $Bms=5$, $Bt1=0.7$, $Bt2=0.05$)



(a) 原圖



(b) 向量圖

Figure 5.11: 結果六($Sms=5$, $St1=0.9$, $St2=0.05$, $Bms=5$, $Bt1=0.9$, $Bt2=0.05$)

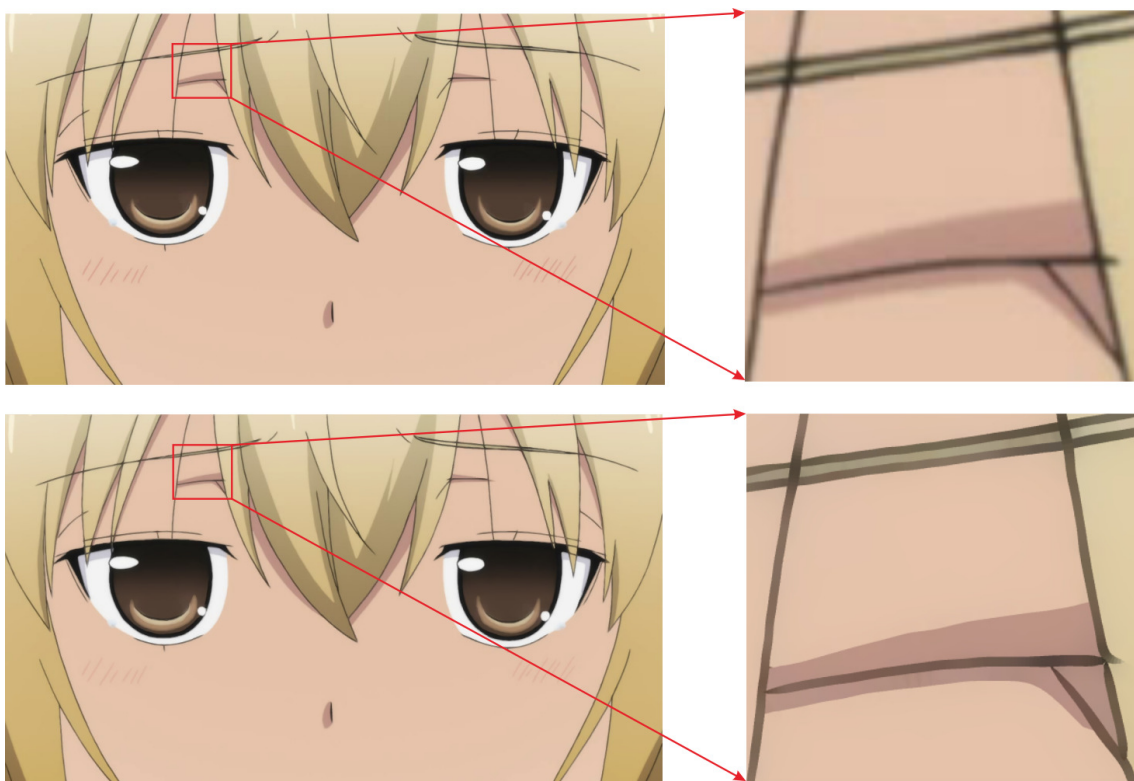


Figure 5.12: 結果七 放大四倍比較圖 ($S_{ms}=5$, $St_1=0.9$, $St_2=0.05$, $B_{ms}=5$, $Bt_1=0.9$, $Bt_2=0.05$)

Chapter 6 結論與未來展望

本論文中我們提出一個可以精準得到筆觸線寬的方法，與如何把斷裂的線縫補起來的方法，提出利用三角化來做區塊分群，而在動畫上面，我們提出一個能維持靜態連續性並找到動態區塊的演算法，與一個簡單的背景重建方法。

我們針對動畫的色塊有很好的效果，但在背景的細節上模糊不清，下面我們列舉了我們目前還不能做的很好的一些情況：

1. 線條複雜的圖，我們的線條會連接錯誤，在可能是線段很接近的地方，我們會有誤判的情況發生，例如圖5.7，右邊胖子左胸的閃電徽章被錯誤的連起來。
2. 動畫部份的結果在區塊細微抖動時有時會被當作靜態場景而不會被偵測出來。
3. 當背景場景有大量細節時，原圖中景深效果的模糊無法萃取，目前的邊界萃取無法做到完美的預處理，導致背景向量化後模糊不清，如圖6.1, 6.2, 6.3, 6.4所示。



(a) 原圖



(b) 向量圖

Figure 6.1: 問題結果一, 士兵的盔甲模糊成一塊, 旗幟上的花紋也模糊不清



Figure 6.2: 問題結果二，上圖為原圖，下圖為向量圖，可以看出都市的細節整個模糊掉了，雲邊緣的細紋也消失了



Figure 6.3: 問題結果三，上圖為原圖，下圖為向量圖，背景樹葉的模糊距離感與原圖相差甚遠



Figure 6.4: 問題結果四，上圖為原圖，下圖為向量圖，能比較出後方兩人的眼睛模糊不清，與原圖差別過大，以及木門牆壁紋理的消失

References

- [1] G. Lecot and B. Lévy, “ARDECO: Automatic Region DEtection and Conversion,” in *17th Eurographics Symposium on Rendering - EGSR'06*, (Nicosia/-Cyprus), pp. 349–360, June 2006.
- [2] J. Sun, L. Liang, F. Wen, and H.-Y. Shum, “Image vectorization using optimized gradient meshes,” Association for Computing Machinery, Inc., July 2007.
- [3] Y.-K. Lai, S.-M. Hu, and R. R. Martin, “Automatic and topology-preserving gradient mesh generation for image vectorization,” *ACM Trans. Graph.*, vol. 28, pp. 85:1–85:8, July 2009.
- [4] Z. Liao, H. Hoppe, D. A. Forsyth, and Y. Yu, “A subdivision-based representation for vector image editing,” *IEEE Trans. Vis. Comput. Graph.*, vol. 18, no. 11, pp. 1858–1867, 2012.
- [5] P. Selinger, “Potrace: a polygon-based tracing algorithm,” 2003.
- [6] S.-H. Zhang, T. Chen, Y.-F. Zhang, S.-M. Hu, and R. R. Martin, “Vectorizing cartoon animations,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 15, no. 4, pp. 618–629, 2009.
- [7] M. ming Cheng, “Curve structure extraction for cartoon images,” 2009.
- [8] C.-C. Weng, H. Chen, and C.-S. Fuh, “A novel automatic white balance method for digital still cameras,” in *ISCAS (4)*, pp. 3801–3804, IEEE, 2005.
- [9] J. Chen, Q. Lei, Y. Miao, and Q. Peng, “Vectorization of line drawing image based on junction analysis,” *SCIENCE CHINA Information Sciences*, vol. 58, no. 7, pp. 1–14, 2015.

- [10] G. Noris, A. Hornung, R. W. Sumner, M. Simmons, and M. Gross, “Topology-driven vectorization of clean line drawings,” *ACM Trans. Graph.*, vol. 32, pp. 4:1–4:11, Feb. 2013.
- [11] B. Whited, B. Whited, J. Rossignac, J. Rossignac, G. Slabaugh, T. Fang, and G. Unal, “Pearling: Stroke segmentation with crusted pearl strings,” in *Proceedings of the 1st International Workshop on Image Mining Theory and Applications IMTA 2008*, pp. 103–112, 2008.
- [12] D. G. Lowe, “Object recognition from local scale-invariant features,” in *Proceedings of the International Conference on Computer Vision-Volume 2 - Volume 2*, ICCV ’99, (Washington, DC, USA), pp. 1150–, IEEE Computer Society, 1999.
- [13] J. Canny, “A computational approach to edge detection,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 8, pp. 679–698, June 1986.
- [14] C. Steger, “An unbiased detector of curvilinear structures,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 20, no. 2, pp. 113–125, 1998.
- [15] D. Comaniciu, P. Meer, and S. Member, “Mean shift: A robust approach toward feature space analysis,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, pp. 603–619, 2002.
- [16] G. Farneback, “Two-frame motion estimation based on polynomial expansion,” in *Image Analysis*, pp. 363–370, Springer, 2003.
- [17] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *Int. J. Comput. Vision*, vol. 60, pp. 91–110, Nov. 2004.
- [18] H. Winnemöller, A. Orzan, L. Boissieux, and J. Thollot, “Texture design and draping in 2d images,” *Computer Graphics Forum (Proceedings of the Eurographics Symposium on Rendering 2009)*, vol. 28, no. 4, pp. 1091–1099, 2009.